

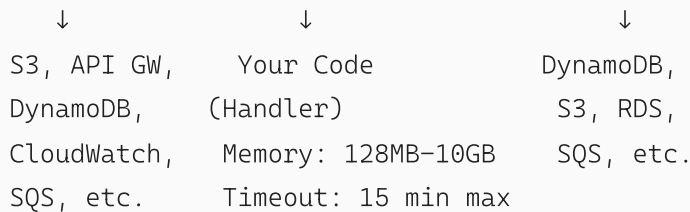
02 AWS Lambda

AWS Lambda

Overview

AWS Lambda is a serverless computing service provided by Amazon Web Services (AWS). It allows you to run code without provisioning or managing servers. AWS Lambda automatically scales your application by running code in response to events, such as changes to data in Amazon S3 buckets or updates in an Amazon DynamoDB table.

Event Sources → Lambda Function → Downstream Services



Serverless Computing

Definition

Serverless computing is a cloud computing execution model where the cloud provider dynamically manages the allocation and provisioning of servers. In a serverless architecture, you only focus on writing code and defining the events that trigger its execution. The cloud provider takes care of the underlying infrastructure, including server management, scaling, and maintenance.

Key Characteristics

Characteristic	Description
No Server Management	Developers do not need to worry about purchasing, provisioning, or managing backend servers.
Event-Driven	Code execution is triggered by events such as HTTP requests, database changes, file uploads, etc.
Automatic Scaling	The service automatically scales with the number of requests or events, handling from a few requests to thousands per second.
Pay-per-Use Billing	You are billed based on the number of executions and the resources consumed, rather than pre-purchased units of capacity.
Built-in High Availability and Fault Tolerance	Serverless platforms provide built-in availability and fault tolerance, ensuring that your applications are highly available and resilient.

Primary Features and Properties of AWS Lambda

Feature	Description
Event-Driven	AWS Lambda is designed to use events like changes to data in an S3 bucket or an update to a DynamoDB table to trigger the execution of code.

Feature	Description
Scaling	Lambda functions can scale automatically by running code in response to each trigger. Your trigger can be an uploaded image, a new log file, a new row in a database, etc.
Languages Supported	AWS Lambda supports multiple programming languages, including Node.js, Python, Ruby, Java, Go, .NET Core, and custom runtimes.
Stateless	By default, AWS Lambda is stateless, meaning each function execution is independent. If you need to maintain state, you would use an external service like Amazon RDS or DynamoDB.
Short-lived	Lambda functions are designed to be short-lived, with a maximum execution time of 15 minutes.
Resource Specification	You can specify the amount of memory allocated to your Lambda function. AWS Lambda allocates CPU power linearly in proportion to the amount of memory configured.
Built-in Fault Tolerance	AWS Lambda maintains compute capacity and infrastructure reliability, including monitoring, logging via Amazon CloudWatch, and automatic retries.
Deployment	Code can be deployed as a Lambda function via a ZIP or JAR file. AWS also provides blueprints to start with sample code for common use cases.
Integrated with AWS Services	AWS Lambda is integrated with many AWS services, making it a flexible tool. For instance, you can trigger a Lambda function from changes in S3, updates in DynamoDB, endpoint requests in API Gateway, etc.
Layers	Lambda Layers are a distribution mechanism for libraries, custom runtimes, and other function dependencies. Layers promote code sharing and separation of responsibilities.
Billing	With AWS Lambda, you are billed for the compute time your code is running. You aren't charged when your code isn't running.
Event Source Mapping	If a Lambda function is triggered by an event source, AWS Lambda takes care of the reading, retries, and deletion of the event, ensuring that each event is processed in order.
Concurrent Executions	AWS Lambda scales functions in parallel. While it manages and scales these automatically, there is a default safety throttle for the number of concurrent executions across all functions in a given region.

Real-World Data Engineering Scenarios

Scenario 1: Real-Time File Processing

Use Case: Automatically process uploaded files, such as images or log files, to extract metadata or transform data.

Implementation:

- **Trigger:** An upload event to an Amazon S3 bucket.
- **Lambda Function:** The function reads the file from S3, processes it (e.g., extracting metadata, transforming data), and stores the results back in S3 or another data store like DynamoDB.
- **Benefits:** Real-time processing, automatic scaling to handle multiple uploads simultaneously, and cost-effective as you only pay for the compute time used.

Scenario 2: Data Transformation and ETL

Use Case: Extract, transform, and load (ETL) data from various sources into a data warehouse for analytics.

Implementation:

- **Trigger:** New data available in an Amazon Kinesis stream or an S3 bucket.
- **Lambda Function:** The function extracts data from the source, applies necessary transformations, and loads it into a data warehouse like Amazon Redshift.
- **Benefits:** Automated data pipelines, reduced operational overhead, and the ability to handle varying data volumes efficiently.

Scenario 3: Automated Backups and Snapshots

Use Case: Automate the process of creating backups or snapshots of databases and other critical data.

Implementation:

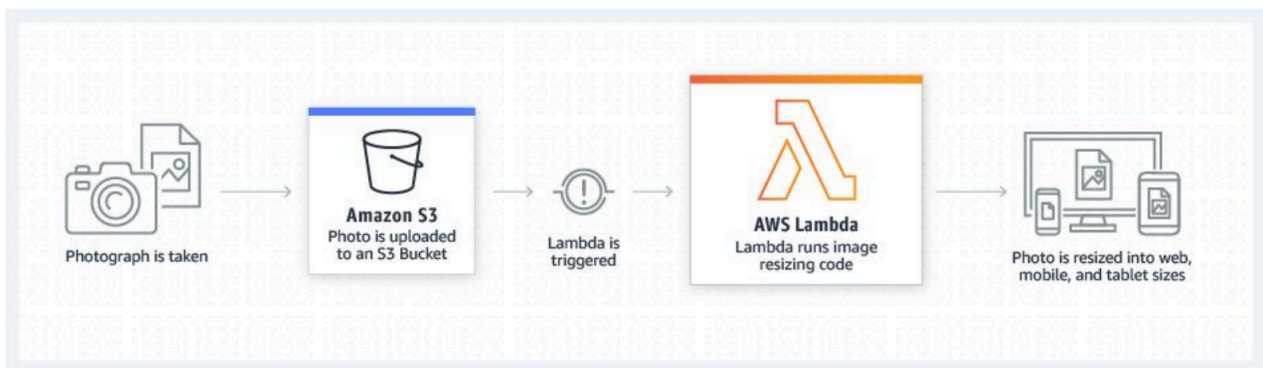
- **Trigger:** A scheduled event using Amazon CloudWatch Events.
- **Lambda Function:** The function creates snapshots of Amazon RDS instances or backups of data stored in DynamoDB and stores them in S3.
- **Benefits:** Ensures regular and reliable backups without manual intervention, enhancing data security and compliance.

Scenario 4: Event-Driven Microservices

Use Case: Build microservices that respond to events in real-time, such as processing orders or updating user profiles.

Implementation:

- **Trigger:** Events from various sources like API Gateway, S3, or DynamoDB Streams.
- **Lambda Function:** The function processes the event, performs necessary business logic, and updates relevant data stores or triggers downstream services.
- **Benefits:** Enables a modular and scalable architecture, faster development cycles, and improved fault isolation.



Conclusion

AWS Lambda and serverless computing offer a powerful and flexible way to build and deploy applications without the need to manage underlying infrastructure. By leveraging AWS Lambda, you can focus on writing code and defining event triggers, while AWS takes care of scaling, availability, and fault tolerance. This makes it an ideal choice for a wide range of data engineering scenarios, from real-time file processing to automated backups and event-driven microservices.